33. Design Xbox Matchmaking

Key Terms:

Term	Full Name	What it does
ELO	Rating system	Chess-derived skill rating (0-3000 scale)
TrueSkill	Microsoft's rating	Bayesian skill rating with uncertainty (μ, σ)
MMR	Matchmaking Rating	Hidden skill score for matchmaking
Ticket	Queue entry	Player's matchmaking request with criteria
RTT	Round-Trip Time	Latency between player and server
Backfill	Mid-match join	Filling spots when players leave

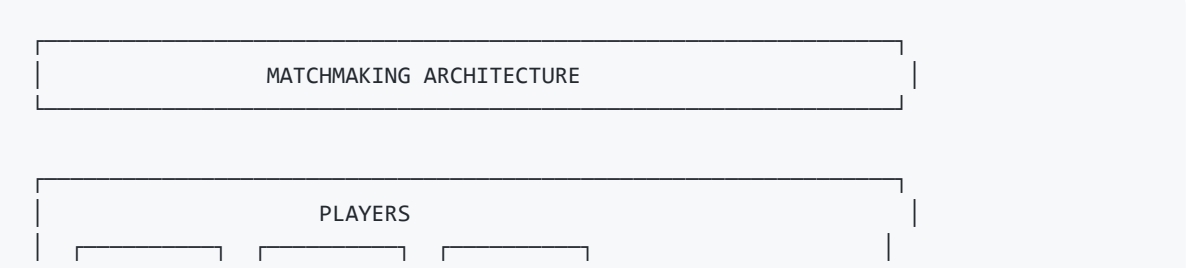
Problem Statement

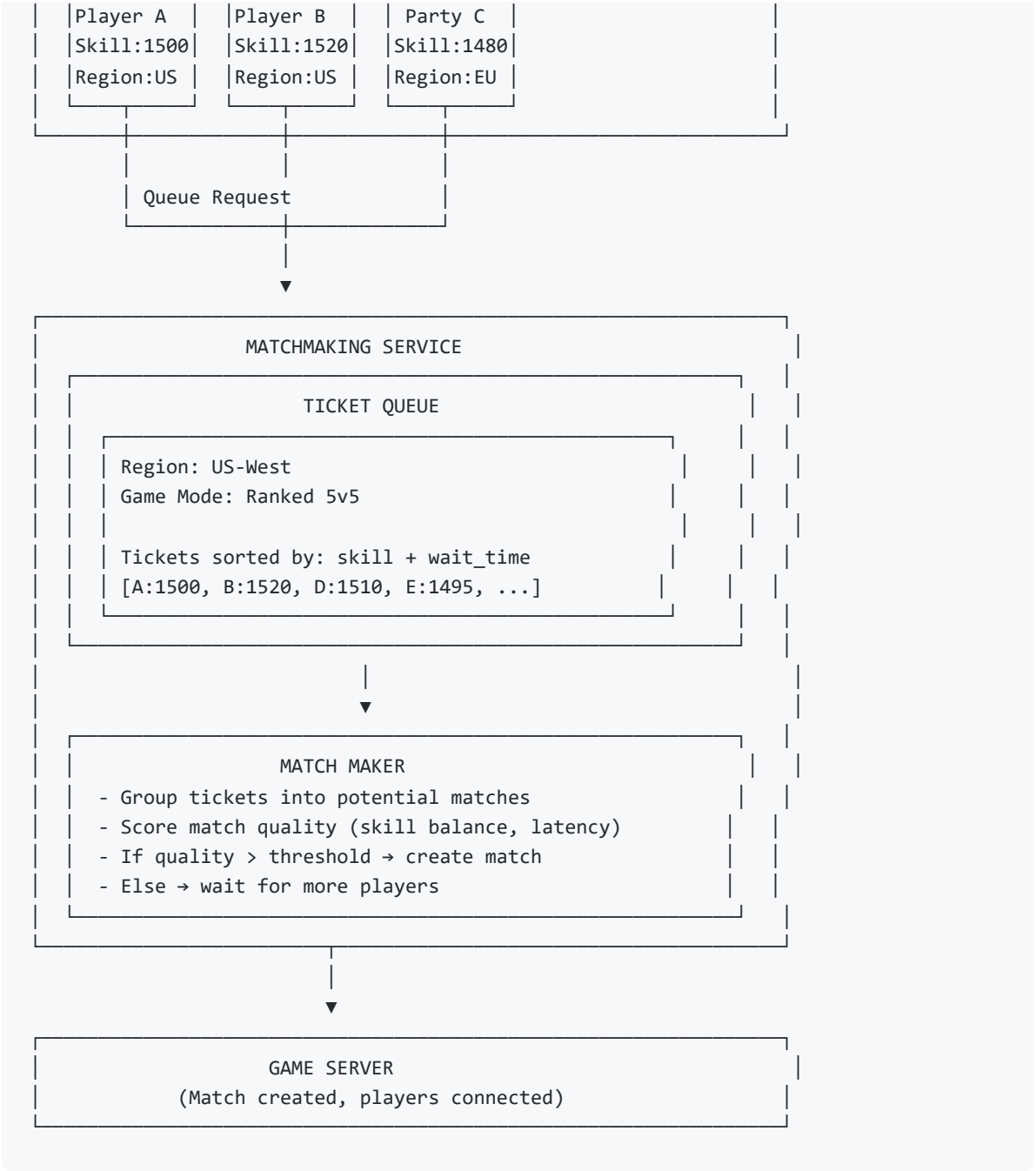
Design a matchmaking system that pairs players based on skill level, latency, and preferences for fair and fast matches.

Clarifying Questions to Ask

1. **Scale**: Concurrent players searching? (1M+ concurrent?)
2. **Match size**: 1v1, teams, battle royale (100 players)?
3. **Criteria**: Skill, latency, party size, game mode?
4. **Wait time**: Max acceptable queue time? (2 minutes?)
5. **Fairness**: How balanced should matches be?
6. **Parties**: Handle groups of friends?
7. **Regions**: Global or region-locked?

High-Level Design





API Design

Matchmaking APIs

Method	Endpoint	Description
`POST`	`/api/v1/matchmaking/queue`	Join matchmaking queue
`DELETE`	`/api/v1/matchmaking/queue/{ticketId}`	Cancel matchmaking
`GET`	`/api/v1/matchmaking/queue/{ticketId}/status`	Get queue status

Queue Request

```
POST /api/v1/matchmaking/queue
{
  "player_id": "player_123",
  "game_mode": "ranked_5v5",
  "skill_rating": 1850,
  "region": "us-west",
  "party": ["player_124", "player_125"],
  "preferences": {
    "max_ping": 80,
    "map_pool": ["map_a", "map_b"]
  }
}

Response: 202 Accepted
{
  "ticket_id": "ticket_789",
  "estimated_wait": 45,
  "queue_position": 12
}
```

Match APIs

Method	Endpoint	Description
`GET`	`/api/v1/matches/{matchId}`	Get match details
`POST`	`/api/v1/matches/{matchId}/accept`	Accept match
`POST`	`/api/v1/matches/{matchId}/results`	Submit match results

WebSocket Events

Event	Direction	Payload
`match.found`	Server → Client	`{ matchId, players, server, map }`
`match.ready`	Server → Client	`{ matchId, allAccepted }`
`queue.update`	Server → Client	`{ position, estimatedWait }`

Key Components

Component	Purpose	Technology

Ticket Service	Create/cancel queue tickets	REST API
Queue Manager	Organize tickets by criteria	Redis
Match Maker	Find optimal player groups	Custom algorithm
Skill Service	Player ratings (ELO/TrueSkill)	Dedicated service
Game Server Allocator	Spin up game servers	Kubernetes
Latency Service	Measure player-to-server RTT	Ping service

Matchmaking Algorithm

```
MATCHMAKING LOOP (runs every 1-2 seconds):
┌─────────────────────────────────────────────────────────────────┐
│  For each queue (game_mode + region):                           │
│                                                                 │
│  1. Get all tickets sorted by (skill, wait_time)                │
│                                                                 │
│  2. For each potential match combination:                       │
│     score = calculate_match_quality(team1, team2)               │
│                                                                 │
│     Match Quality Score:                                        │
│     ┌─────────────────────────────────────────────────────────┐ │
│     │ skill_balance = 1 - |avg(team1) - avg(team2)| / 500   | │
│     │ wait_penalty = avg_wait_time / max_wait_time          | │
│     │ latency_score = 1 - avg_latency / max_acceptable      | │
│     │                                                       | │
│     │ score = 0.5 * skill_balance                           | │
│     │       + 0.3 * wait_penalty    (longer wait = less picky)| │
│     │       + 0.2 * latency_score                           | │
│     └─────────────────────────────────────────────────────────┘ │
│                                                                 │
│  3. If score > threshold (e.g., 0.7):                           │
│     - Remove tickets from queue                                 │
│     - Create match                                              │
│     - Allocate game server                                      │
│     - Notify players                                            │
│                                                                 │
│  4. Tickets waiting > 2 min: expand skill range                 │
└─────────────────────────────────────────────────────────────────┘
```

Skill Rating Systems

```
ELO RATING:
┌─────────────────────────────────────────────────────────────────┐
│  New Rating = Old Rating + K * (Actual - Expected)              │
│                                                                 │
```

```
│  Expected = 1 / (1 + 10^((opponent - player) / 400))          │
│                                                               │
│  Example:                                                     │
│    Player: 1500, Opponent: 1600                               │
│    Expected win probability: 36%                              │
│    If player wins: +23 points                                 │
│    If player loses: -13 points                                │
│                                                               │
│  Pros: Simple, well-understood                                │
│  Cons: Doesn't handle team games well                         │
└──────────────────────────────────────────────────────────────┘

TRUESKILL (Microsoft's system):
┌──────────────────────────────────────────────────────────────┐
│  Rating = (μ, σ) where:                                       │
│    μ = skill estimate (mean)                                  │
│    σ = uncertainty (standard deviation)                       │
│                                                               │
│  Conservative estimate = μ - 3σ                               │
│                                                               │
│  After each game:                                             │
│    - Winners' μ increases, σ decreases                        │
│    - Losers' μ decreases, σ decreases                         │
│    - Upsets cause larger updates                              │
│                                                               │
│  Pros: Handles teams, converges faster                        │
│  Cons: More complex                                           │
└──────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Sharding**: Separate queues per region and game mode
- **Ticket expiry**: Auto-remove stale tickets (timeout)
- **Party handling**: Treat party as single ticket, match by highest skill
- **Server allocation**: Pre-warm game servers during peak hours
- **Backfill**: Fill spots when players leave mid-match

Trade-offs

Decision	Option A	Option B	Recommendation
Skill rating	ELO	TrueSkill	**TrueSkill** - better for teams
Match speed	Fast (loose)	Slow (strict)	**Adaptive** - expand over time
Queue	Global	Regional	**Regional** - lower latency
Party matching	By avg skill	By highest	**Highest** - fairer for opponents

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Ticket queue and player state management	Tech Guide
AKS/Kubernetes	Game server allocation and scaling	Tech Guide

How to Explain It Well

> **Opening**: "Matchmaking balances match quality with wait time. I'll use TrueSkill for ratings and adaptive matching that loosens criteria over time."

> **Key talking points**:
>
> - "TrueSkill tracks skill (μ) and uncertainty (σ) - better for team games"
> - "Match quality score combines skill balance, wait time, and latency"
> - "Adaptive: after 2 minutes, expand acceptable skill range"
> - "Shard queues by region for low latency matches"
> - "Pre-warm game servers during peak hours"
